Problem 5

In [23]:
```
from ISLP import load_data
Carseats = load_data('Carseats')
```

In [24]:
```
# 5a
from sklearn.model_selection import train_test_split
from ISLP.models import ModelSpec as MS
import numpy as np

model = MS(Carseats.columns.drop('Sales'), intercept=False)
D = model.fit_transform(Carseats)
feature_names = list(D.columns)
X = np.asarray(D)
y = Carseats['Sales'].values

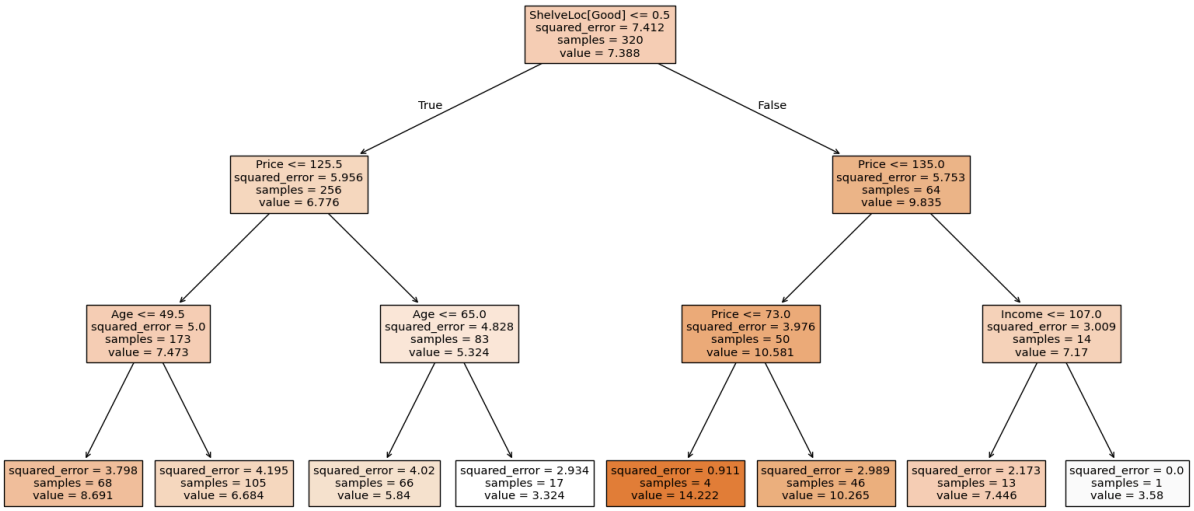
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [25]:
```
# 5b
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt

tree = DecisionTreeRegressor(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

plt.figure(figsize=(20, 10))
plot_tree(tree, filled=True, feature_names=feature_names)
plt.show()

print(f"Test MSE: {np.mean((y_test - tree.predict(X_test))**2):.4f}")
```




```
Test MSE: 6.6841
```

In [26]:
```
# 5c
from sklearn.model_selection import GridSearchCV, KFold

tree_full = DecisionTreeRegressor(random_state=42)
tree_full.fit(X_train, y_train)
mse_full = np.mean((y_test - tree_full.predict(X_test))**2)

ccp_path = tree_full.cost_complexity_pruning_path(X_train, y_train)

kfold = KFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
    param_grid={'ccp_alpha': ccp_path.ccp_alphas},
    cv=kfold,
    scoring='neg_mean_squared_error'
)
grid.fit(X_train, y_train)

tree_pruned = grid.best_estimator_
mse_pruned = np.mean((y_test - tree_pruned.predict(X_test))**2)

print(f"Test MSE (full): {mse_full:.4f}")
print(f"Test MSE (pruned): {mse_pruned:.4f}")
```

```
Test MSE (full): 6.0762
Test MSE (pruned): 5.9015
```

In [27]:
```
# 5d
from sklearn.ensemble import RandomForestRegressor
import pandas as pd

bagging = RandomForestRegressor(
    n_estimators=100,
    max_features=X_train.shape[1],
    random_state=42
)
bagging.fit(X_train, y_train)

mse_bagging = np.mean((y_test - bagging.predict(X_test))**2)
print(f"Test MSE (bagging): {mse_bagging:.4f}")

(
    pd.DataFrame(
        {'importance': bagging.feature_importances_}, index=feature_names
    )
    .sort_values('importance', ascending=False)
)
```

```
Test MSE (bagging): 3.0638
```

Out[27]:

	importance
Price	0.308711
ShelveLoc[Good]	0.198396
Age	0.119995
CompPrice	0.092736
Advertising	0.082817
ShelveLoc[Medium]	0.064620
Income	0.050830
Population	0.039730
Education	0.031198
Urban[Yes]	0.006382
US[Yes]	0.004585

In [28]:

```
# 5e
forest = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
forest.fit(X_train, y_train)

mse_forest = np.mean((y_test - forest.predict(X_test))**2)
print(f"Test MSE (forest): {mse_forest:.4f}")

print(
    pd.DataFrame(
        {'importance': forest.feature_importances_}, index=feature_names
    )
    .sort_values('importance', ascending=False)
)

print("\nEffect of m (max_features) on test MSE:")
for m in [1, 3, 6, X_train.shape[1]]:
    rf = RandomForestRegressor(
        n_estimators=100,
        max_features=m,
        random_state=42
    )
    rf.fit(X_train, y_train)
    mse = np.mean((y_test - rf.predict(X_test))**2)
    print(f"  m={m}: Test MSE = {mse:.4f}")
```

```
Test MSE (forest): 3.0638
                   importance
Price                0.308711
ShelveLoc[Good]      0.198396
Age                  0.119995
CompPrice            0.092736
Advertising          0.082817
ShelveLoc[Medium]    0.064620
Income               0.050830
Population           0.039730
Education            0.031198
Urban[Yes]           0.006382
US[Yes]              0.004585

Effect of m (max_features) on test MSE:
  m=1: Test MSE = 6.0147
  m=3: Test MSE = 3.9052
  m=6: Test MSE = 3.0847
  m=11: Test MSE = 3.0638
```

Problem 6

In [29]:

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

np.random.seed(42)
X = np.random.uniform(-2, 2, (100, 2))
y = (X[:, 0]**2 + X[:, 1]**2 <= 2).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

def plot_decision_boundary(model, ax, title):
    xx, yy = np.meshgrid(np.arange(-2, 2, 0.02), np.arange(-2, 2, 0.02))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdYlBu')
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap='RdYlBu')
    ax.set_title(title)

models = {
    'Linear SVC': SVC(kernel='linear', random_state=42),
    'Polynomial SVM (deg=2)': SVC(kernel='poly', degree=2, random_state=42),
    'Radial SVM': SVC(kernel='rbf', random_state=42)
}

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for i, (name, model) in enumerate(models.items()):
    model.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, model.predict(X_train))
```

```
    test_acc = accuracy_score(y_test, model.predict(X_test))
    title = f"{name}: Train Error - {1 - train_acc:.2f} Test Error - {1 - test_acc:
    plot_decision_boundary(model, axes[i], title)

plt.tight_layout()
plt.show()
```

